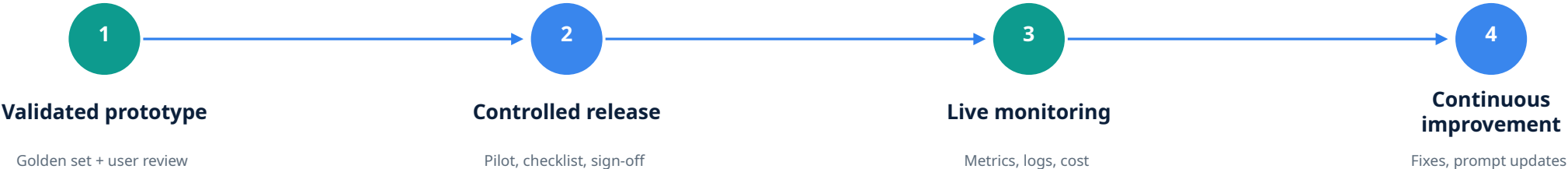


From tested prototype to reliable operation

After the workflow passed evaluation, I defined how it would be released, monitored, recovered and improved.

Link to previous step: 07 - Evaluation & Testing confirmed prompt behavior, routing quality, cost and failure cases. 08 explains how I would operate the workflow safely after release.

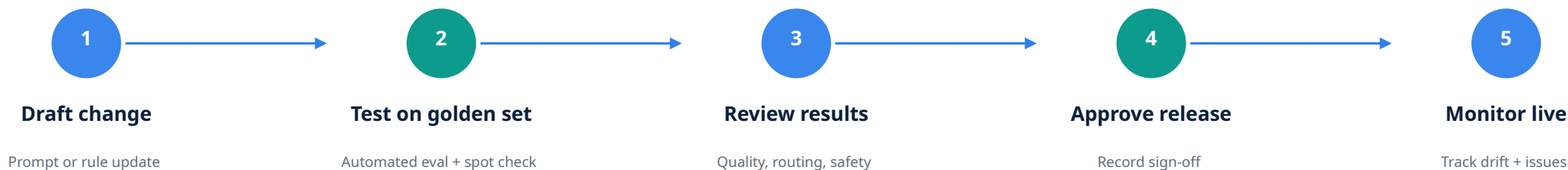


Release governance and version control

I treated prompts, guardrails and routing rules like product assets, not loose text pasted into a workflow.

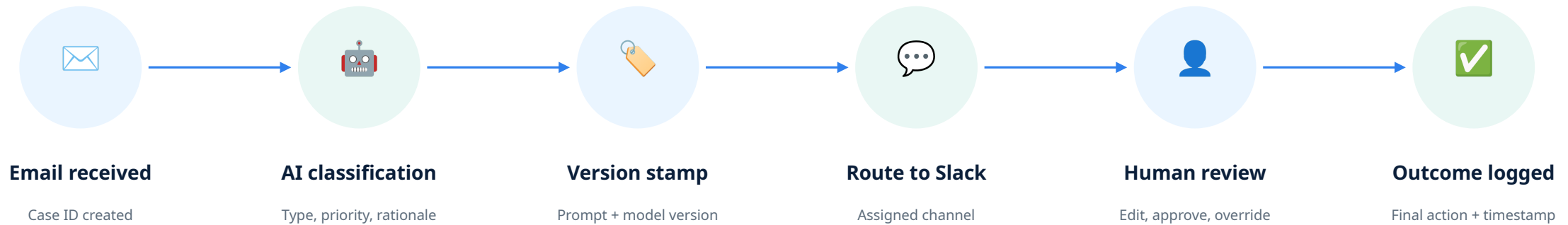
Why this mattered

Small changes can alter routing, tone and cost. Every change needed a record and review.



Traceability and audit trail

Every AI decision was traceable from the original email to the reviewer action and final outcome.



What was logged

- Email subject / sender
- Classification and confidence
- Prompt version and channel
- Reviewer action and final outcome



Why it mattered

- Easy audit trail
- Better root-cause analysis
- Reusable evidence for improvements



Practical example

- Misrouted email spotted
- Prompt version traced
- Fix tested and redeployed

Monitoring live performance

The operating question was not only "is it running?" It was "is it accurate, stable and cost-efficient?"

Classification accuracy

92%

ILLUSTRATIVE

Exception rate

4%

ILLUSTRATIVE

Median latency

38 sec

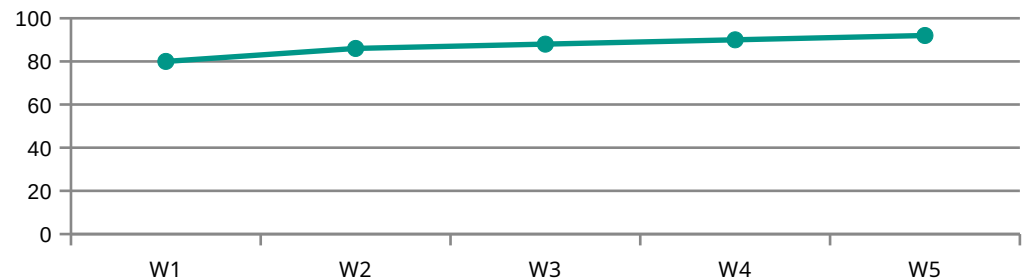
ILLUSTRATIVE

Token spend

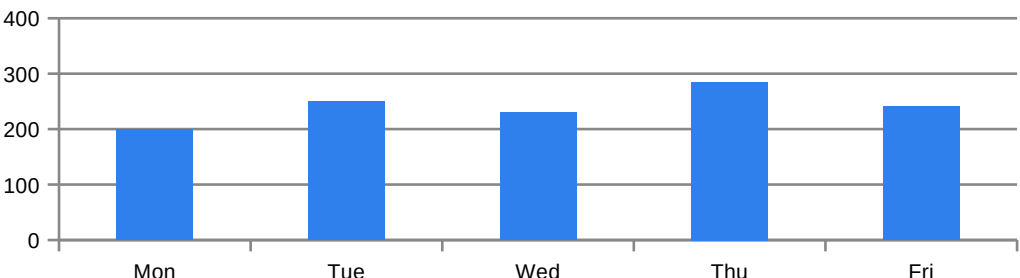
\$0.42/day

ILLUSTRATIVE

Weekly accuracy trend



Volume by day



Weekly review loop

Low-confidence and misrouted cases

Latency spikes or failed runs

Credit and token usage changes

Incident detection and recovery

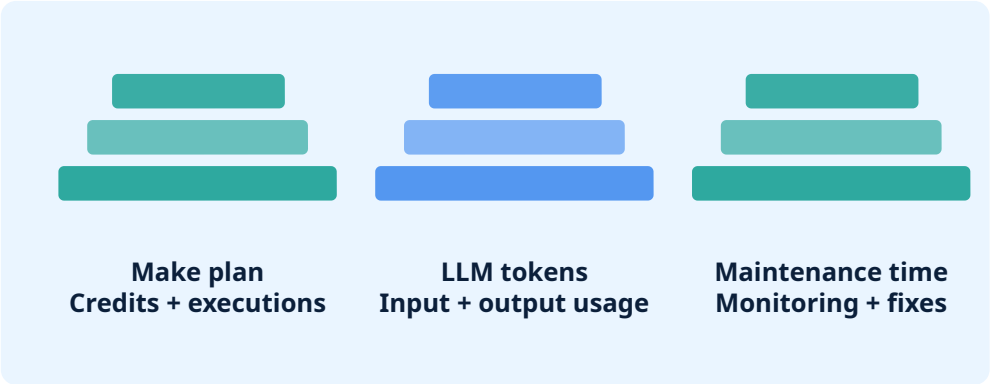
I treated failures as operational events with detection, triage, recovery and prevention steps.



Issue	How detected	Immediate response	Outcome
API failure	Failed execution log	Retry or pause scenario	faster recovery fewer repeats
Misroute spike	Slack/user feedback	Rollback prompt version	
Cost spike	Daily budget alert	Switch model or shorten prompt	

Operating cost control and maintenance

I separated platform cost, model cost and maintenance effort so the workflow stayed affordable and governable.



Plan choice logic

Core (\$9)	Early testing
Pro (\$16)	Single-owner production workflow
Teams (\$29)	Multiple reviewers and governance

How I monitored token cost



Usage logging

- Tokens per run
- Daily and monthly totals



Budget controls

- Usage alerts
- High-cost prompt review



Optimization

- Compare model options
- Shorten prompts
- Reduce unnecessary retries

Cost references: Make public list prices for 10k credits/month. Workflow usage figures should be treated as illustrative until production logs are available.